

LangChain AI Agents — Advanced Notes

Topics 12–18: Agent State, Memory, Agentic RAG, Human-in-the-Loop, Multi-Agent Systems, Evaluation, and Production Deployment

Goal: Understand how a basic tool-calling agent evolves into a reliable, observable, production-ready AI system.

12. Agent State

Agent state is the information an agent carries while a task is executing. It can contain messages, the current task, tool results, retrieved documents, intermediate decisions, and status flags.

Mental model: **State = everything the workflow needs to continue from its current point.** Explicit state makes multi-step agents easier to pause, resume, retry, inspect, and debug.

Simple state model

```
from typing import TypedDict

class AgentState(TypedDict):
    messages: list
    user_name: str
    task_complete: bool
```

Stateful workflow example

```
from typing import TypedDict

class State(TypedDict):
    question: str
    answer: str
    count: int

def process(state: State):
    state["count"] += 1
    state["answer"] = "Processed: " + state["question"]
    return state

state = {"question": "What is RAG?", "answer": "", "count": 0}
state = process(state)
print(state)
```

In LangGraph-style applications, nodes read and update a shared state. This is especially useful when an agent has several steps or branches.

13. Memory

Memory is information retained beyond the immediate tool call. State normally represents the current workflow context; memory commonly refers to information retained across turns or sessions.

Types of memory

Type	Purpose	Example
Short-term	Current conversation	Previous messages
Long-term	Persist useful facts	User preferences in DB
Semantic	Retrieve related knowledge	Vector database
Episodic	Remember past events	Previous support task

Conversation memory

```
from langchain_core.messages import HumanMessage, AIMessage
```

```

messages = [
    HumanMessage(content="My name is Ankit."),
    AIMessage(content="Nice to meet you, Ankit."),
    HumanMessage(content="What is my name?")
]

# The model can use the history to answer:
# "Your name is Ankit."

```

Production note: Do not store everything blindly. Consider privacy, retention, deletion, access control, and whether the information is actually useful.

14. Agentic RAG

Traditional RAG usually follows: question → retrieve → generate. **Agentic RAG** lets the agent decide whether retrieval is needed, which source to use, whether evidence is sufficient, and whether another search is necessary.

Comparison

Normal RAG:
 Question -> Retriever -> Documents -> LLM -> Answer

Agentic RAG:
 Question -> Agent
 |
 +--> Search
 |
 +--> Is evidence enough?
 | |
 No Yes
 | |
 Search again Answer

Retrieval tool example

```

from langchain_core.tools import tool

@tool
def search_docs(query: str) -> str:
    """Search the knowledge base for relevant information."""
    # Replace this with your vector-store retriever.
    return "Relevant document text for: " + query

```

In a real project, connect the tool to a vector store such as Chroma, FAISS, Qdrant, Pinecone, or PostgreSQL/pgvector. Track sources and citations so the final answer can be grounded in evidence.

15. Human-in-the-Loop

Human-in-the-loop (HITL) means the agent pauses for human approval or input before a sensitive action continues. It is useful for irreversible, expensive, externally visible, or high-risk actions.

Approval flow

```

User -> Agent -> Plan action -> Human approval
                        /           \
                    Yes             No
                    |               |
                Execute         Stop/revise

```

Example: approval before sending email

```

from langchain_core.tools import tool

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email after human approval."""
    # Real implementation would call an email API.
    return f"Email sent to {to}"

```

```

draft = {
    "to": "client@example.com",
    "subject": "Project Update",
    "body": "The project is ready for review."
}

approved = input("Send this email? [y/n]: ")

if approved.lower() == "y":
    print(send_email.invoke(draft))
else:
    print("Email was not sent.")

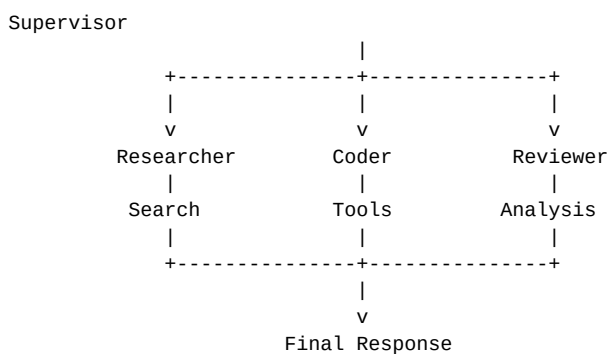
```

For production, represent approval as durable workflow state so the system can pause and later resume. Avoid relying only on terminal input.

16. Multi-Agent Systems

A multi-agent system uses multiple specialized agents instead of one agent doing everything. Specialization can improve organization and make complex workflows easier to maintain.

Architecture



Conceptual example

```

class ResearchAgent:
    def run(self, task):
        return "Research findings"

class CodingAgent:
    def run(self, task):
        return "Implementation"

class ReviewAgent:
    def run(self, task):
        return "Review findings"

research = ResearchAgent().run("Find requirements")
code = CodingAgent().run(research)
review = ReviewAgent().run(code)

print(review)

```

In real LangChain projects, orchestration can be implemented with LangGraph. A supervisor/router decides which specialist should handle the next step.

17. Evaluation

An agent that works once is not necessarily reliable. Evaluation measures whether it completes tasks correctly, selects tools properly, retrieves useful evidence, stays grounded, and remains efficient and safe.

Important evaluation dimensions

Dimension	Question
-----------	----------

Task success	Did the agent complete the task?
Correctness	Is the final answer correct?
Tool selection	Did it choose the right tool?
Arguments	Were tool arguments correct?
Retrieval	Was useful evidence retrieved?
Groundedness	Does the answer follow the evidence?
Latency	How long did the run take?
Cost	How many model/tool calls were made?
Safety	Did it avoid unsafe actions?

Simple evaluation dataset

```
test_cases = [
    {"question": "What is 20 * 5?", "expected": "100"},
    {"question": "What is 10 + 7?", "expected": "17"},
]

def evaluate(agent):
    results = []

    for case in test_cases:
        response = agent.invoke({
            "messages": [
                {"role": "user", "content": case["question"]}
            ]
        })

        results.append({
            "question": case["question"],
            "expected": case["expected"],
            "response": response,
        })

    return results
```

For serious systems, build a representative evaluation dataset and use tracing/evaluation tooling. Test both final answers and intermediate behavior such as tool calls and retrieval.

18. Production Deployment

A notebook prototype is different from a production agent. Production systems need security, reliability, observability, scalability, cost controls, and safe access to tools.

Architecture

```
Frontend
|
v
FastAPI / Django
|
v
Agent Service
|
+----- LLM
+----- Tools ---- DB / APIs / Search
+----- Memory --- DB / Vector DB
|
v
Logs + Traces + Evaluation
```

Minimal FastAPI wrapper

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

@app.post("/chat")
def chat(request: ChatRequest):
    # Replace this with your real agent.invoke(...) call.
    result = f"Agent received: {request.message}"
    return {"response": result}

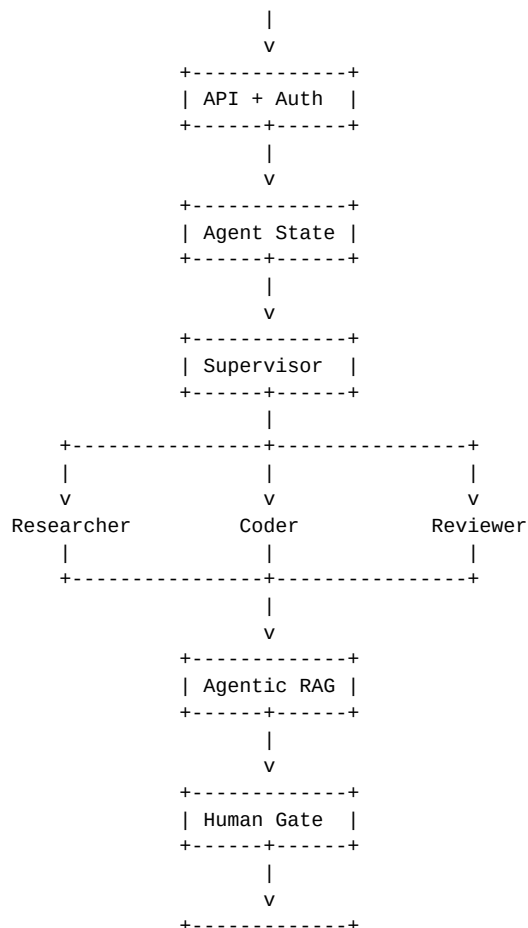
```

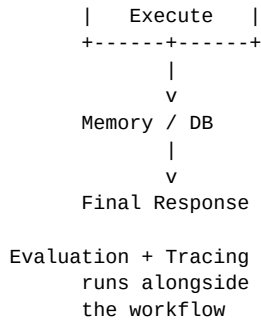
Production checklist

1. **Secrets:** Keep API keys in environment variables or a secret manager.
2. **Authentication:** Authenticate users before accessing private data.
3. **Authorization:** Restrict tools according to user/agent permissions.
4. **Tool safety:** Validate arguments and require confirmation for sensitive operations.
5. **Timeouts and retries:** Handle slow or temporarily failing providers.
6. **Rate limits:** Control abuse and unexpected API costs.
7. **Observability:** Trace model calls, tool calls, latency, failures, and token usage.
8. **Evaluation:** Run regression tests whenever prompts, tools, or models change.
9. **Deployment:** Containerize the API/worker and deploy it on suitable infrastructure.
10. **Data protection:** Encrypt sensitive data and enforce least-privilege access.

Putting 12–18 Together

USER





Key Takeaways

Agent State: Current information required to continue a workflow.

Memory: Information retained for future turns or tasks.

Agentic RAG: The agent decides when and how to retrieve information.

Human-in-the-loop: A human approves or modifies important actions.

Multi-agent: Specialized agents collaborate through an orchestrator.

Evaluation: Measure correctness, task success, tools, retrieval, cost, latency, and safety.

Production: Add authentication, authorization, guardrails, observability, reliability, and deployment controls.

Core formula: LLM + Tools + State + Loop + Memory + Retrieval + Guardrails + Evaluation + Deployment = Production-grade Agent System.